



GitHub Actions



A simple CI/CD guide you'll save and reuse

Save • Share • Build Better Pipelines



Why GitHub Actions?

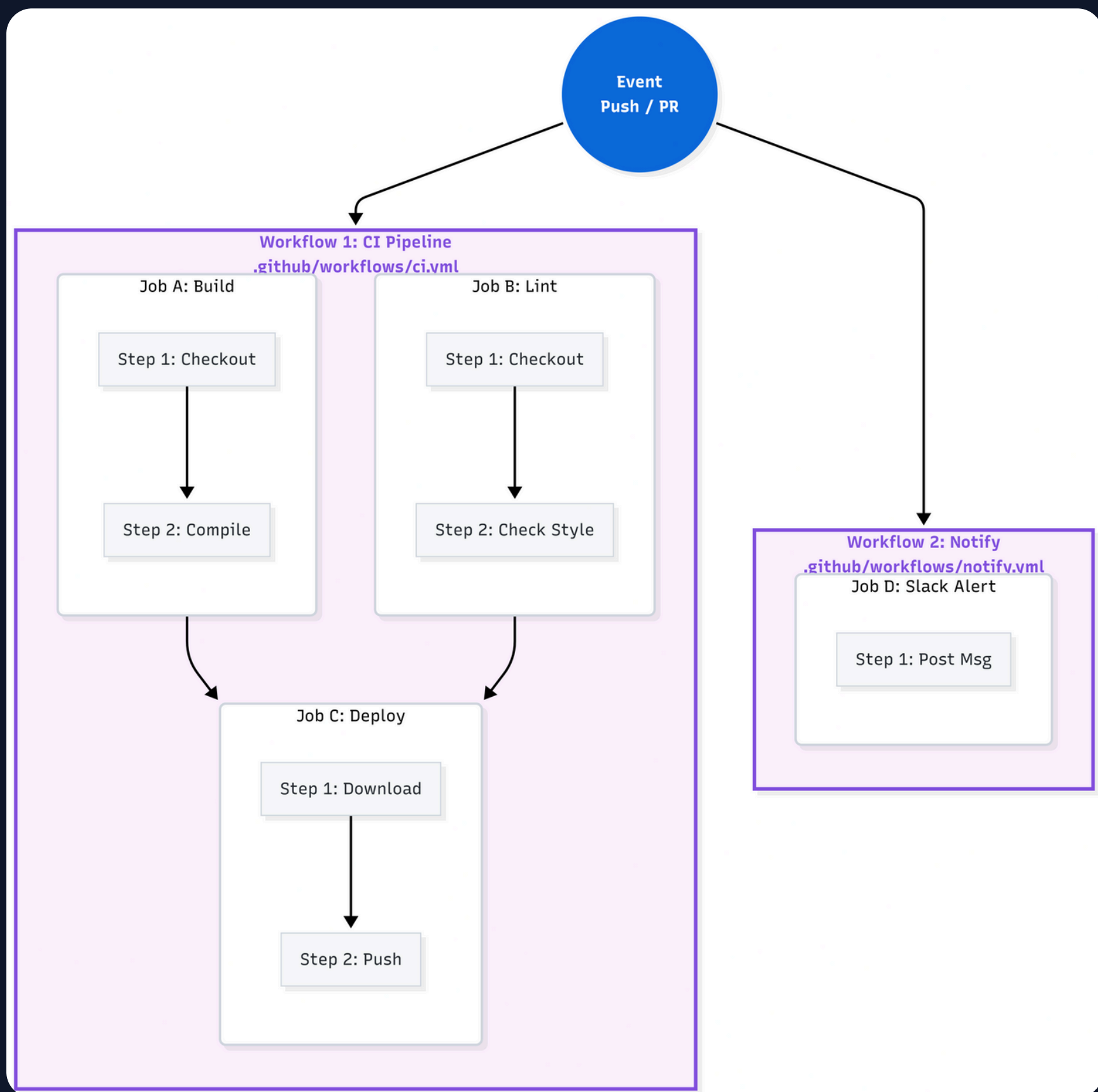
- Built into GitHub with no extra CI/CD tools required
- Event-driven automation using push, pull requests, schedules, or manual triggers
- YAML-based workflows stored alongside application code
- Native integration with repositories, PRs, and issues
- Reusable actions with secure secrets management
- Flexible runners for build, test, and deployment workflows

[GitHub Actions](#) | [Official Documentation](#)



How it works?

- **Event:** The trigger (e.g., Push, PR).
- **Workflows:** The automation pipelines triggered by the event.
- **Jobs:** Groupings of tasks within a workflow (running in parallel or sequence).
- **Steps:** The individual commands or actions that make up a job.





Anatomy of a Workflow YAML File

- GitHub Actions Workflows are defined as YAML (YML) files and must be placed inside the following directory within the repository:

.github/workflows/

- The workflow file name does not matter. As long as the YAML file is valid and located in this path, GitHub Actions will automatically detect it and trigger the workflow based on the configured event.

```
.github > workflows > demo1.yml
1 | # Workflow name - appears in GitHub Actions tab
2 | name: Events Demo 1
3 |
4 | # EVENT TRIGGERS - Define when this workflow runs
5 | on:
6 |   # Trigger on pull requests to main branch (only when opened)
7 |   pull_request:
8 |     branches:
9 |       - main
10 |     types:
11 |       - opened # Only when PR is first created, not on updates
12 |
13 | # Manual trigger - Run workflow from GitHub UI (Actions tab → Run workflow button)
14 | workflow_dispatch:
15 |
16 | # JOBS - Define the work to be done (jobs run in parallel by default)
17 | jobs:
18 |   deploy:
19 |     # Runner environment - Virtual machine to execute this job
20 |     # Options: ubuntu-latest, windows-latest, macos-latest
21 |     runs-on: ubuntu-latest
22 |
23 |     # STEPS - Sequential tasks within this job
24 |     steps:
25 |       # Output event data for debugging
26 |       # ${ } = expression syntax, toJSON() = converts object to JSON
27 |       - name: Output event data...
28 |
29 |
30 |       # Clone repository code to the runner
31 |       # 'uses' = run a pre-built action, @v3 = version 3
32 |       - name: Get code...
33 |
34 |
35 |       # Install Node.js dependencies
36 |       # npm ci = clean install from package-lock.json (faster than npm install)
37 |       - name: Install dependencies...
38 |
39 |
40 |       # Run test suite - workflow fails if tests fail
41 |       - name: Test code...
42 |
43 |
44 |       # Build production bundle (creates dist/build folder)
45 |       - name: Build code...
46 |
47 |
48 |       # Deploy (placeholder - replace with actual deployment action/commands)
49 |       - name: Deploy project...
```



A simple workflow file typically includes the following sections:

- **Workflow Name**
 - Defined using the `name` keyword. This name appears in the Actions tab of the GitHub repository.
- **Event Trigger**
 - Defined using the `on` keyword. Common triggers include:
 - `workflow_dispatch` – allows manual execution from the Actions tab
 - `pull_request` – triggered when a pull request is opened or updated
 - `push` – triggered when code is pushed to specified branches
- **Jobs**
 - Defined using the `jobs` keyword. Each job represents a unit of work.
 - The job name (for example, `deploy`) can be any meaningful identifier.
- **Runner Environment**
 - Defined using the `runs-on` keyword. This specifies the environment where the job executes (for example, `ubuntu-latest`).
- **Execution Steps**
 - Defined using the `steps` keyword. Steps are executed sequentially within a job.
 - Each step typically includes:
 - `name` – *human-readable description*
 - `run` or `uses` – *command or action to execute*
 - `id` – *optional, useful for referencing step outputs*

```
# a simple workflow

name: Java Maven Workflow

on: [push]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: mvn clean install
```



Actions

Actions are repetitive tasks that are predefined either via custom approach or available in the official or community action.

The actions are used using the `uses` keyword and refer the action identifier. Refer the image above where the checkout action is referred.

Can also specify a version as the code behind these get updated over time.

For example, to download code from the repo, the Checkout action can be used from the marketplace.

Link to marketplace: <https://github.com/marketplace/actions/checkout>

The screenshot shows the GitHub Actions Marketplace page for the 'Checkout' action. The page header includes the GitHub logo, 'Marketplace / Actions', a search bar, and navigation icons. The main content area features the 'Checkout' action card with a 'Build and Test' badge showing 'passing'. The card lists three versions: 'Checkout v6', 'Checkout v5', and 'Checkout v4'. Each version has a 'What's new' section. The 'About' section on the right describes the action as 'Checkout a Git repository at a particular version', shows the current version 'v6.0.1' as 'Latest', and lists tags like 'utilities'. It also shows a list of contributors and resources like 'Start a discussion', 'Open an issue', and 'Pull requests'.

Marketplace / Actions

Checkout (Actions)

Build and Test **passing**

Checkout v6

What's new

- Improved credential security: `persist-credentials` now stores credentials in a separate file under `$RUNNER_TEMP` instead of directly in `.git/config`
- No workflow changes required — `git fetch`, `git push`, etc. continue to work automatically
- Running authenticated git commands from a [Docker container action](#) requires Actions Runner [v2.329.0](#) or later

Checkout v5

What's new

- Updated to the node24 runtime
 - This requires a minimum Actions Runner version of [v2.327.1](#) to run.

Checkout v4

This action checks-out your repository under `$GITHUB_WORKSPACE`, so your workflow can access it.

About

Checkout a Git repository at a particular version

v6.0.1 **Latest**

By [actions](#)

Verified ✓

GitHub has manually verified the creator of the action as an official partner organization. For more info see [About badges in GitHub Marketplace](#).

Tags 1

[utilities](#)

Contributors 62

+ 48 contributors

Resources

- Start a discussion
- Open an issue 519
- Pull requests 116



Contexts

Collection of variables to access information about workflow runs, variables, runner environments, jobs, and steps. Each context is an object that contains properties, which can be strings or other objects.

Context	Description	Sample YAML Use Case
github	Provides information about the repository, branch, event, and actor	<pre>if: github.ref == 'refs/heads/main'</pre>
env	Accesses environment variables defined at workflow, job, or step level	<pre>run: echo "Env is \${ env.APP_ENV }"</pre>
secrets	Provides secure access to encrypted secrets	<pre>run: docker login -u \${ secrets.DOCKER_USER } -p \${ secrets.DOCKER_PASSWORD }</pre>
steps	Accesses outputs and status of previous steps in the same job	<pre>run: echo "Version \${ steps.build.outputs.version }"</pre>
jobs	Accesses results and outputs of other jobs in the workflow	<pre>if: jobs.build.result == 'success'</pre>
needs	Ensures job dependencies and accesses dependent job results	<pre>needs: build ; if: needs.build.result == 'success'</pre>
runner	Provides information about the runner executing the job	<pre>if: runner.os == 'Linux'</pre>
matrix	Provides values for parallel job execution	<pre>run: echo "Running on Java \${ matrix.java }"</pre>

<https://docs.github.com/en/actions/reference/workflows-and-actions/contexts>



Expressions

Expressions add conditional logic and dynamic values to workflows. They are used in conditions, environment variables, and input evaluation.

Syntax

`${{ expression }}`

Common Expression Examples

Run only on main branch

```
if: github.ref == 'refs/heads/main'
```

Run step after successful build

```
if: steps.build.conclusion == 'success'
```

Run job only if tests pass

```
if: needs.test.result == 'success'
```

Set environment dynamically

```
APP_ENV: ${{{ github.ref == 'refs/heads/main' && 'prod' || 'dev' }}
```

Check runner OS

```
if: runner.os == 'Linux'
```

Use manual workflow inputs

```
if: inputs.environment == 'staging'
```




Trivia

How to run jobs in a sequence?

Jobs run in parallel by default. To run jobs sequentially, use the `needs` keyword to define dependencies.

Example:

```
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - run: echo "Build job"
  deploy:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - run: echo "Deploy job"
```

What are the different workflow triggers?

Workflow triggers define when a workflow starts. They are configured using the `on` keyword.

Common triggers include:

`push`, `pull_request`, `workflow_dispatch`, `schedule`

Example:

```
on:
  push:
    branches:
      - main
  pull_request:
  workflow_dispatch:
```

Ways to **cancel** a workflow?

A workflow can be canceled manually from the Actions tab. It can also be canceled automatically using concurrency.

Example:

```
concurrency:
  group: production-deploy
  cancel-in-progress: true
```

This cancels any in-progress workflow when a new one starts in the same group.

How to **skip** a workflow?

Workflows can be skipped by including specific keywords in the commit message.

Skip keywords:

`[skip ci]` or `[ci skip]`

Example commit message:

`git commit -m "Update README [skip ci]"`

This prevents the workflow from running on push.



Workflow Triggers

Event Filters

Used to narrow when a workflow triggers for an event.

Common filters include branches, tags, paths, or types of activity within an event.

They control scope without writing extra conditions in jobs.

Event Filters = where / which files / branches

```
# workflow triggers on a push event to the 'main' branch
on:
  push:
    branches:
      - main
    paths:
      - 'src/**'
```

Activity Types

Used with specific events to trigger workflows only for certain sub-types of that event.

Often applies to pull_request, issues, or check_run.

Controls the type of action inside the event, not the branch or file paths.

Activity Types = what action within the event

```
# workflow triggers on a pull request activity
on:
  pull_request:
    types: [opened, reopened, synchronize]
```



Secrets

Store sensitive information in organization, repository or repository secrets. These values are stored such that no one sees them. Can be set at org , environment & repo level.

The screenshot shows the GitHub Actions settings page for a repository named 'gh-actions-react-demo-2' under the user 'nelsonselvam'. The page is titled 'Actions secrets and variables'. It features a left sidebar with navigation options: General, Access (Collaborators, Moderation options), Code and automation (Branches, Tags, Rules, Actions, Models, Webhooks, Copilot, Environments, Codespaces, Pages), Security (Advanced Security, Deploy keys, Secrets and variables), and Actions. The main content area is divided into 'Environment secrets' and 'Repository secrets'. The 'Environment secrets' section shows a message 'This environment has no secrets.' with a 'Manage environment secrets' button. The 'Repository secrets' section has a 'New repository secret' button and a table listing secrets. The table has columns for 'Name' and 'Last updated'. It contains one entry: 'API_TOKEN' with a lock icon, the text 'now', and edit/delete icons.

Repository secret added.

Actions secrets and variables

Secrets and variables allow you to manage reusable configuration data. Secrets are **encrypted** and are used for sensitive data. [Learn more about encrypted secrets](#). Variables are shown as plain text and are used for **non-sensitive** data. [Learn more about variables](#).

Anyone with collaborator access to this repository can use these secrets and variables for actions. They are not passed to workflows that are triggered by a pull request from a fork.

Environment secrets

This environment has no secrets.

[Manage environment secrets](#)

Repository secrets

[New repository secret](#)

Name	Last updated
API_TOKEN	now

To retrieve the secrets, use the expression syntax with the `secrets` context.

`${{ secrets.<SECRET_NAME> }}`



Environment Variables

Environment Variables can be defined in the workflow file at different levels: top workflow level, job level and step level using the `env` keyword.

To retrieve the variable value:

- Use the \$ prefixed environment variable name: **`$<VARIABLE_NAME>`**
- Use expression syntax with `env` context: **`${{ env.<VARIABLE_NAME> }}`**

```
name: Greeting on variable day

on:
  workflow_dispatch

env:
  DAY_OF_WEEK: Monday

jobs:
  greeting_job:
    runs-on: ubuntu-latest
    env:
      Greeting: Hello
    steps:
      - name: "Say Hello Mona it's Monday"
        run: echo "$Greeting $First_Name. Today is $DAY_OF_WEEK!"
        env:
          First_Name: Mona
```

To define configuration variable for multiple workflows at the org, repo or environment level, use the Settings page to define them at the respective levels

The screenshot shows the GitHub Actions Settings page for a repository named 'gh-actions-react-demo-2'. The left sidebar contains navigation links for various settings categories. The main content area is titled 'Actions secrets and variables' and is divided into two tabs: 'Secrets' and 'Variables'. The 'Variables' tab is active, showing a section for 'Environment variables' with a message stating 'This environment has no variables.' and a button to 'Manage environment variables'. Below this, there is a section for 'Repository variables' which includes a table with columns for Name, Value, and Last updated. The table contains one entry: 'ENV_VAR' with the value 'env_var_value' and the last updated time 'now'. A 'New repository variable' button is located to the right of the table.

Name	Value	Last updated
ENV_VAR	env_var_value	now



Repository Environments

Attach them to repos to provide additional configuration variables & secrets by creating different environments.

The screenshot shows the GitHub repository settings page for 'Actions secrets and variables'. The left sidebar contains navigation links: General, Access (Collaborators, Moderation options), Code and automation (Branches, Tags, Rules, Actions, Models, Webhooks, Copilot, Environments, Codespaces, Pages), Security (Advanced Security, Deploy keys, Secrets and variables), and Actions. The main content area is titled 'Actions secrets and variables' and includes a description of secrets and variables. It features two tabs: 'Secrets' and 'Variables'. Under 'Secrets', there are two sections: 'Environment secrets' and 'Repository secrets'. The 'Environment secrets' section has a 'Manage environment secrets' button and a table with columns 'Name', 'Environment', and 'Last updated'. It lists two secrets: 'MONGODB_PASSWORD' and 'MONGODB_USERNAME', both associated with the 'testing' environment and updated 'last week'. The 'Repository secrets' section has a 'New repository secret' button and a similar table with columns 'Name' and 'Last updated'. It also lists 'MONGODB_PASSWORD' and 'MONGODB_USERNAME', both updated 'last week', with edit and delete icons.

```
name: Deployment
on:
  push:
    branches:
      - main
      - dev
  workflow_dispatch:
env: # env level variables
  MONGODB_DB_NAME: gha-demo
jobs:
  test:
    environment: testing
    runs-on: ubuntu-latest
    env: # job level variables
      MONGODB_CLUSTER_ADDRESS: cluster0.pjzlfj8.mongodb.net
      MONGODB_USERNAME: ${ secrets.MONGODB_USERNAME } # secrets context object
      MONGODB_PASSWORD: ${ secrets.MONGODB_PASSWORD }
      PORT: 3000
    steps:
      - name: Get Code
        uses: actions/checkout@v3
      - name: Cache dependencies
        uses: actions/cache@v3
```



Custom Actions

Create our own actions so that we can use them beyond using shell commands and marketplace.

Problem:

- YAML gets messy
- Logic repeats
- Workflows become unreadable novels

Solution:

👉 Custom Actions = reusable, versioned, sharable logic

Think functions, not scripts.

GitHub gives us three flavors:

JavaScript Actions

Docker Actions

Composite Actions

Each with a different personality.



JavaScript Actions (Fast & Native)

Best for:

- API calls
- GitHub context processing
- Lightweight logic

Key traits:

- Runs on Node.js
- No container build
- Fast execution
- Tight GitHub integration

Docker Actions (Maximum Power)

Best for:

- Custom runtimes
- CLI tools
- Language-specific environments

Key traits:

- Full OS control
- Any language
- Heavier, slower startup

Composite Actions (YAML Magic)

Best for:

- Reusing workflow steps
- Standardizing pipelines
- No code required

Key traits:

- Pure YAML
- Fast
- No runtime logic
- Easy to maintain



Best Practices

- ✓ Keep workflows readable
- ✓ Extract repetition into reusable actions
- ✓ Start small and iterate often
- ✓ Treat actions like libraries, not one-off scripts

Common Mistakes

- ✗ Copy-pasting workflows across repos
- ✗ Hardcoding secrets
- ✗ Skipping version tags
- ✗ Overusing Docker for simple tasks

Final Takeaway

- Workflows automate tasks.
- Actions automate teams.

Remember

-  Build once
-  Reuse everywhere
-  Let automation quietly do the work

Prepared by Nelson Selvam

Empowering developers to automate, simplify, and reuse workflows with GitHub Actions

LinkedIn: [linkedin.com/in/nelsonselvam](https://www.linkedin.com/in/nelsonselvam)

GitHub: github.com/nelsonselvam